Sprawozdanie

Projektowanie scalonych systemów wbudowanych

Projekt licznika

Elektronika i Telekomunikacja, 2 rok

Jakub Lasak

05.01.2025

1. Cel ćwiczenia

Celem ćwiczenia było zaprojektowanie licznika synchronicznego, który liczy w NKB od 0 do 30. Gdy wejście jest równe 0, licznik zlicza do góry zwiększając wartość o 1, a gdy wejście jest równe 1, zmienia kierunek i zlicza w dół co dwa. W ramach ćwiczenia przygotowano krótki testbench i uruchomiono symulację, aby potwierdzić poprawne działanie.

2. Budowa układu

Licznik jest synchronicznym układem sekwencyjnym pracującym w naturalnym kodzie binarnym (NKB). Zawiera jeden rejestr 5-bitowy (przechowuje wartość q) oraz prostą logikę następnego stanu, która zależnie od sygnału sterującego (we) wybiera inkrementację o +1 lub dekrementację o −2 z poprawnym „zawijaniem" zakresu od 0 do 30. Reset jest asynchroniczny, aktywny stanem niskim.

Porty modułu licznika:

- **Wejścia**
  - clk – zegar
  - rst_n – reset asynchroniczny, aktywny 0; natychmiast ustawia licznik do 0
  - we – kierunek/tryb pracy: 0 → w górę o +1, 1 → w dół o −2
- **Wyjście**
  - q[4:0] – aktualna wartość licznika

Bloki funkcjonalne i zasada działania:

- **Rejestr stanu (q)**:
  - gdy !rst_n → q <= 5'd0 (asynchroniczne wyzerowanie, bez czekania na zegar).
  - w przeciwnym razie q przyjmuje wartość policzoną przez logikę kierunku.
- **Logika kierunku zliczania:**
  - UP (we = 0, inkrementacja o 1)
    Jeśli q == 30 → przejście na 0 (zawinięcie 30→0); w pozostałych przypadkach q <= q + 1
  - DOWN (we = 1, dekrementacja o 2)
    Jeśli q >= 2 → q <= q - 2.
    Dla brzegów: q==1 → 29, q==0 → 30 (zachowanie w zbiorze {0…30})

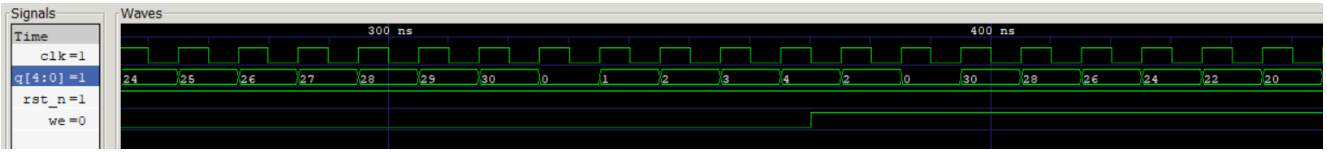
3. Kod w Verilogu

```
`timescale 1ns/1ps

module pssw305_counter (
    input  wire       clk,
    input  wire       rst_n,
    input  wire       we,
    output reg  [4:0] q
);
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            q <= 5'd0;
        end else if (we == 1'b0) begin
            if (q == 5'd30) q <= 5'd0;
            else            q <= q + 5'd1;
        end else begin
            if (q >= 5'd2)       q <= q - 5'd2;
            else if (q == 5'd1)  q <= 5'd29;
            else                 q <= 5'd30;
    end
endmodule

`timescale 1ns/1ps

module tb_pssw305_counter;
  reg        clk;
  reg        rst_n;
  reg        we;
  wire [4:0] q;

  pssw305_counter dut (.clk(clk), .rst_n(rst_n), .we(we), .q(q));

  initial clk = 1'b0;
  always #5 clk = ~clk;

  initial begin

    $dumpfile("pssw305.vcd");
    $dumpvars(0, tb_pssw305_counter);
    $display(" we  q(dec) q(bin)");
    $monitor(" %0d     %2d     %05b", we, q, q);

    rst_n = 0; we = 0;
    #17; rst_n = 1;

    repeat (35) @(posedge clk);

```

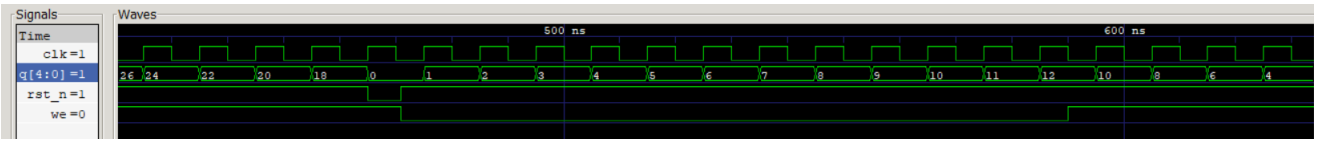
```
50.         @(negedge clk) we = 1;
51.         repeat (10) @(posedge clk);
52.
53.         rst_n = 0; #6; rst_n = 1;
54.         we = 0;
55.         repeat (12) @(posedge clk);
56.
57.         @(negedge clk) we = 1;
58.         repeat (8) @(posedge clk);
59.
60.         @(negedge clk) we = 0;
61.         repeat (10) @(posedge clk);
62.
63.         rst_n = 0; #4; rst_n = 1;
64.         we = 0;
65.         repeat (10) @(posedge clk);
66.
67.         $finish;
68.       end
69.     endmodule
70.
```

4. Wyniki

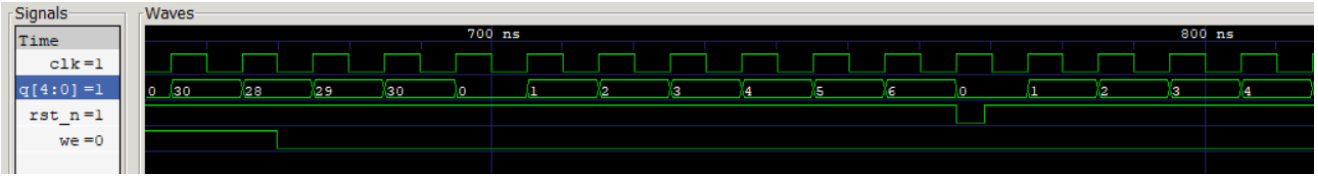
Poniżej wklejone są trzy zrzuty ekranu z przykładowymi fragmentami symulacji przedstawiające działanie układu:



Rysunek 1 Zrzut ekranu działania licznika - zmiana kierunku



Rysunek 2 Zrzut ekranu działania licznika - reset asynchroniczny



Rysunek 3 Zrzut ekranu działania licznika - praca w górę po resecie

Na pierwszym zrzucie ekranu widzimy zliczanie w górę (o 1) do 30 przy wejściu ustawionym na 0 (we = 0), po przełączeniu wejścia na 1 (we = 1) zliczanie w dół (o 2). Na drugim zdjęciu możemy zauważyć impuls resetu (rst_n =0), który natychmiast ustawia q=0, po zwolnieniu resetu i ustawieniu we = 0 licznik rośnie: 0,1,2,3,… Na ostatnim zdjęciu widoczne jest przejście automatycznie z 30 na 0 i dalsze zliczanie w trybie we = 0.